

PPT Roguelite — Proyecto de aprendizaje Godot

Banco de pruebas para SPQR: Punic Wars

Título de trabajo, sin cerrar. No hace falta nombre definitivo para un proyecto instrumental.

0. Propósito

Este no es un juego a publicar. Es un **ejercicio de aprendizaje de Godot** con un criterio de diseño muy concreto: cada sistema que se construya aquí debe practicar algo que luego se necesita en SPQR. Si un sistema no transfiere, se corta, aunque "quedaría bien".

Referentes de mercado ya existentes (Rogueshambo, Rock Paper Roguelike, Handmancers) confirman que el concepto es viable en poco alcance — no hay que inventar nada nuevo, solo construirlo con cabeza propia para aprender el motor.

1. Concepto (pitch corto)

Un roguelite de combates 1v1 resueltos por piedra-papel-tijera(s), con una progresión de dos monedas (una permanente entre partidas, otra temporal dentro de cada run) y una estructura de mapa por nodos. Sin arte, sin narrativa, sin sistemas decorativos: solo lo estructural.

2. Bucle de juego (core loop)

1. El jugador entra a una run desde un hub.
2. Recorre un mapa de nodos simple (combate / elite / descanso / jefe).
3. Cada combate se resuelve por rondas de PPT contra un enemigo con un patrón de IA propio.
4. Ganar una run o morir termina la run: lo temporal se pierde, lo permanente se queda.
5. Vuelve al hub, gasta la moneda permanente en mejoras fijas, empieza otra run.

Esto es, estructuralmente, el mismo esqueleto que SPQR (mapa de nodos → combate → Gloria permanente en el hub → Oro temporal en la run), a una escala que se puede terminar en semanas, no en meses.

3. Sistema de combate

Base (obligatoria): piedra-papel-tijera clásico de 3 elementos, resuelto por rondas. Cada ronda ambos eligen en ciego, se resuelve el ganador, se aplica daño.

Extensión opcional (fase avanzada, no obligatoria): ampliar a 5 elementos estilo piedra-papel-tijera-lagarto-Spock, con una matriz de contras circular — esto es una réplica casi literal del pentagrama RPSLS que ya tienes cerrado para SPQR. Si llegas a esta fase, ya habrás practicado exactamente la lógica de resolución que necesitas para el combate real.

CERRADO — estructura del combate: no es un lanzamiento único por nodo. Cada enemigo tiene una barra de vida, y se juegan rondas de PPT en ciego hasta que la vida de uno de los dos llega a 0. El que gana una ronda hace daño; empate no hace daño. Esto es deliberado y no un capricho: un solo lanzamiento por nodo se sentiría como cara-o-cruz, no como combate, y además no practicarías nada del bucle de turnos ni del manejo de HP que necesitas para SPQR (donde el combate real es exactamente esto: una barra que aguanta a tope hasta que llega a 0).

Dimensiona la vida del enemigo para que el combate dure pocas rondas (orientativo: 3-5), ni una tirada

suelta ni una guerra de desgaste de 20 rondas – cifra exacta a sentir jugando, no a calcular en papel.

Reglas mínimas a definir en Godot (no en el papel, directamente en código, para aprender iterando):

- Daño mínimo garantizado por golpe que impacta (ya tienes este principio cerrado en SPQR – cópialo aquí)
- Orden de resolución si hay modificadores (¿mitigación plana antes o después de la porcentual?)

4. Economía (dos monedas, igual que Gloria/Oro)

	Permanente (entre runs)	Temporal (dentro de la run)
Nombre placeholder	"Chispa"	"Cobre"
Se pierde al morir	No	Sí
Para qué sirve	Desbloques fijos en el hub (nuevos patrones de IA	Comprar/mejorar dentro de la run (ej. un "comodín" de un uso, o una

	Permanente (entre runs)	Temporal (dentro de la run)
	disponibles como enemigos, mejoras base)	ventaja de una ronda)
Analogía SPQR	Gloria	Oro

No hace falta una tercera moneda. Si en algún momento se te ocurre añadir una, es la señal de que te estás complicando de más — para este ejercicio, dos monedas son más que suficientes para practicar la separación conceptual.

5. Progresión permanente

Mantenlo a nivel de estructura, no de contenido:

- Un puñado de mejoras fijas compradas con la moneda permanente (ej. +1 de vida base, desbloquear un enemigo nuevo en el pool, desbloquear el comodín temporal)
- Sin árbol de habilidades ramificado — eso ya lo tienes en SPQR y no aporta aprendizaje nuevo de Godot, solo tiempo de más

- El objetivo aquí es el **guardado de datos entre sesiones** (persistencia), no la profundidad del árbol
-

6. Estructura de run / mapa

Mapa lineal o ligeramente ramificado (no hace falta el grafo complejo de SPQR): 5-8 nodos, tipos mínimos (Combate, Descanso, Jefe). Esto es suficiente para practicar generación simple de estructura de run sin construir un sistema de mapa completo.

7. Qué NO construir (para no perder el foco de aprendizaje)

- Arte propio (usa formas geométricas o texto: círculo=piedra, cuadrado=papel, triángulo=tijera)
 - Música o sonido elaborado
 - Sistema de mazos/cartas tipo Handmancers (esa complejidad no aporta nada nuevo a SPQR)
 - Más de un personaje jugable
 - Balanceo numérico fino — usa cifras de prueba y sigue adelante, exactamente como haces en SPQR
-

8. Hoja de ruta por fases (con objetivo Godot explícito)

Fase	Qué se construye	Qué se aprende de Godot
0	Motor de resolución PPT puro, sin UI (prints en consola)	Lógica de nodos, scripts, señales básicas
1	Combate jugable con UI mínima (botones + barra de vida)	Control nodes, máquina de estados de turno (Fase Jugador → Resolución → Fase Enemigo)
2	Guardado de la moneda permanente entre sesiones	Resource/FileAccess, Autoload/singleton para datos persistentes
3	Mapa de nodos simple +	Gestión de escenas, transición entre pantallas

Fase	Qué se construye	Qué se aprende de Godot
	hub con mejoras compradas	
4 (opcional)	Extensión a 5 elementos (RPSLS)	Practicar la resolución de matriz de contras — transferencia directa al pentagrama de SPQR
5 (opcional)	Patrones de IA enemiga (aleatorio, telegráfico, contra tu última jugada)	Primeras nociones de comportamiento/IA simple, útil para el diseño de enemigos de SPQR

No hace falta llegar a la fase 5 para que el ejercicio haya cumplido su función. Las fases 0-3 ya cubren el 80% del aprendizaje necesario para atacar el primer prototipo digital de SPQR.

9. Tabla de transferencia directa a SPQR

Sistema aquí	Sistema equivalente en SPQR
Resolución PPT / RPSLS	Pentagrama de contras entre las 5 clases
Moneda permanente + temporal	Gloria + Oro
Guardado entre sesiones	Persistencia de Gloria y desbloques entre runs
Máquina de estados de turno	Fase Jugador / Fase Enemigos ya cerrada en SPQR
Mapa de nodos simple	Grafo de campaña por acto
Patrones de IA enemiga	Comportamiento de las clases enemigas por acto

10. Siguiendo paso inmediato

Empezar por la Fase 0: el motor de resolución, sin interfaz, solo para aprender a estructurar un proyecto Godot desde cero. No hace falta diseño de pantallas

todavía — mismo criterio que ya aplicaste en SPQR
(papel antes que pixel).